

Vox Self-Driving Music Playlist: A Z Specification

Formal Model of vox-1rxb (eager fill, auto-advance, rotate-at-12)

July 2026

Contents

1	Introduction	2
2	Basic Types	2
3	Free Types	2
4	Global Constants	2
5	State	2
6	Initialisation	3
7	Operations	3
7.1	TurnOn	3
7.2	FillProduces	4
7.3	Advance (auto-advance and manual skip)	4
7.4	VibeChange	5
7.5	TurnOff	5
8	Key Properties	5
9	Findings from Formalisation	6

1 Introduction

This specification pins the target behaviour of the vox background-music playlist for a single (*vibe*, *style*) pool. On turn-on the daemon generates the first track and starts playing it, then generates the remaining tracks in the background (one at a time) up to a fixed pool size. Playback advances automatically as each track ends; once the pool is full, playback rotates among the saved tracks with no further generation. The model exists to make the transitions and invariants unambiguous before implementation, and to expose any underspecified or contradictory cases (see Section 9).

2 Basic Types

A track is an opaque saved-audio identifier (in the implementation, the content-addressed filename within a pool).

$[TRACK]$

3 Free Types

The playback mode is a four-state machine.

$Mode ::= off \mid generatingFirst \mid playingFilling \mid playingRotating$

A Z boolean (avoids the ProB/B keyword clash with `BOOL`).

$ZBOOL ::= ztrue \mid zfalse$

4 Global Constants

The pool fills to **maxPool** distinct tracks, after which generation stops and playback rotates. The design value is 12.

$maxPool : \mathbb{N}$
$maxPool = 12$

5 State

`playing` and `lastPlayed` are modelled as sets of at most one track (an “optional track”): empty when nothing is playing / no track has yet been played.

Playlist

$pool : \mathbb{P} \text{ TRACK}$
 $playing : \mathbb{P} \text{ TRACK}$
 $lastPlayed : \mathbb{P} \text{ TRACK}$
 $mode : \text{Mode}$
 $filling : \text{ZBOOL}$

$\#pool \leq \text{maxPool}$
 $\#playing \leq 1$
 $\#lastPlayed \leq 1$
 $playing \subseteq pool$
 $lastPlayed \subseteq pool$
 $mode = \text{off} \Rightarrow (playing = \emptyset \wedge filling = \text{zfalse})$
 $mode = \text{generatingFirst} \Rightarrow$
 $(pool = \emptyset \wedge playing = \emptyset \wedge filling = \text{ztrue})$
 $mode = \text{playingFilling} \Rightarrow$
 $(\#playing = 1 \wedge \#pool \geq 1$
 $\wedge \#pool < \text{maxPool} \wedge filling = \text{ztrue})$
 $mode = \text{playingRotating} \Rightarrow$
 $(\#playing = 1 \wedge \#pool = \text{maxPool} \wedge filling = \text{zfalse})$
 $filling = \text{ztrue} \Rightarrow$
 $(mode = \text{generatingFirst} \vee mode = \text{playingFilling})$

6 Initialisation

The daemon starts idle with an empty pool. **InitFromDisk** models a restart where a pool of already-saved tracks is loaded (still idle until turned on).

Init

Playlist'

$pool' = \emptyset$
 $playing' = \emptyset$
 $lastPlayed' = \emptyset$
 $mode' = \text{off}$
 $filling' = \text{zfalse}$

InitFromDisk

Playlist'
 $diskPool? : \mathbb{P} \text{ TRACK}$

$\#diskPool? \leq \text{maxPool}$
 $pool' = diskPool?$
 $playing' = \emptyset$
 $lastPlayed' = \emptyset$
 $mode' = \text{off}$
 $filling' = \text{zfalse}$

7 Operations

7.1 TurnOn

From **off**, enter the pool. An empty pool begins generating the first track; a partial pool starts playing and resumes the background fill; a full pool starts playing and rotates. (Restart is **InitFromDisk** followed by **TurnOn**.)

$TurnOn$ $\Delta Playlist$
$mode = off$ $pool' = pool$ $lastPlayed' = \emptyset$ $(pool = \emptyset \Rightarrow$ $\quad (mode' = generatingFirst \wedge playing' = \emptyset$ $\quad \wedge filling' = ztrue))$ $(\#pool \geq 1 \wedge \#pool < maxPool \Rightarrow$ $\quad (mode' = playingFilling \wedge playing' \subseteq pool$ $\quad \wedge \#playing' = 1 \wedge filling' = ztrue))$ $(\#pool = maxPool \Rightarrow$ $\quad (mode' = playingRotating \wedge playing' \subseteq pool$ $\quad \wedge \#playing' = 1 \wedge filling' = zfalse))$

7.2 FillProduces

The background fill delivers one fresh track. It is enabled only while **filling** and the pool is not yet full, and the track must be new (the deterministic collision-free naming from `bas7` guarantees this). The first delivered track starts playback; reaching `maxPool` stops the fill and switches to rotation.

$FillProduces$ $\Delta Playlist$ $newTrack? : TRACK$
$filling = ztrue$ $\#pool < maxPool$ $newTrack? \notin pool$ $pool' = pool \cup \{newTrack?\}$ $lastPlayed' = lastPlayed$ $(mode = generatingFirst \Rightarrow$ $\quad (playing' = \{newTrack?\} \wedge mode' = playingFilling$ $\quad \wedge filling' = ztrue))$ $(mode = playingFilling \Rightarrow$ $\quad (playing' = playing$ $\quad \wedge (\#pool' = maxPool \Rightarrow$ $\quad \quad (mode' = playingRotating \wedge filling' = zfalse))$ $\quad \wedge (\#pool' < maxPool \Rightarrow$ $\quad \quad (mode' = playingFilling \wedge filling' = ztrue))))$

7.3 Advance (auto-advance and manual skip)

When the current track ends, move to another track, avoiding an immediate repeat of the just-ended track when the pool holds more than one. With a single track (the transient state before the second track lands) it replays — the only intended looping. This one transition serves both the automatic track-end advance and the manual `/music next`.

Advance
 $\Delta\text{Playlist}$

$(mode = \text{playingFilling} \vee mode = \text{playingRotating})$
 $pool' = pool$
 $mode' = mode$
 $filling' = filling$
 $lastPlayed' = playing$
 $(\#pool \geq 2 \Rightarrow$
 $\quad (playing' \subseteq pool \wedge \#playing' = 1$
 $\quad \wedge playing' \neq playing))$
 $(\#pool = 1 \Rightarrow playing' = playing)$

$\text{TrackEnds} \hat{=} \text{Advance}$
 $\text{Skip} \hat{=} \text{Advance}$

7.4 VibeChange

A vibe/style change finishes the current song, then switches to the new pool (the tracks already saved on disk for the new key). The selection of initial mode/playing/filling mirrors **TurnOn** applied to the new pool. **newPool?** is the new key's on-disk track set.

VibeChange

$\Delta\text{Playlist}$

$newPool? : \mathbb{P} \text{ TRACK}$

$mode \neq \text{off}$
 $\#newPool? \leq \text{maxPool}$
 $pool' = newPool?$
 $lastPlayed' = \emptyset$
 $(newPool? = \emptyset \Rightarrow$
 $\quad (mode' = \text{generatingFirst} \wedge playing' = \emptyset$
 $\quad \wedge filling' = \text{ztrue}))$
 $(\#newPool? \geq 1 \wedge \#newPool? < \text{maxPool} \Rightarrow$
 $\quad (mode' = \text{playingFilling} \wedge playing' \subseteq newPool?$
 $\quad \wedge \#playing' = 1 \wedge filling' = \text{ztrue}))$
 $(\#newPool? = \text{maxPool} \Rightarrow$
 $\quad (mode' = \text{playingRotating} \wedge playing' \subseteq newPool?$
 $\quad \wedge \#playing' = 1 \wedge filling' = \text{zfalse}))$

7.5 TurnOff

Cancel the background fill and stop playback. The pool remains on disk.

TurnOff

$\Delta\text{Playlist}$

$mode \neq \text{off}$
 $mode' = \text{off}$
 $playing' = \emptyset$
 $lastPlayed' = \emptyset$
 $filling' = \text{zfalse}$
 $pool' = pool$

8 Key Properties

The following hold by construction (state invariants) or as operation postconditions; they are the intended proof obligations for animation:

- **Bounded pool:** $\#pool \leq \text{maxPool}$ always.
- **Generation only below full:** `FillProduces` requires $\#pool < \text{maxPool}$; at $\#pool = \text{maxPool}$ the mode is `playingRotating` with $\text{filling} = \text{zfalse}$, so no generation occurs.
- **At most one fill:** $\text{filling} = \text{ztrue}$ implies mode is `generatingFirst` or `playingFilling`.
- **Playing is real:** when $\text{mode} \neq \text{off}$, $\#playing = 1$ and $\text{playing} \subseteq \text{pool}$.
- **No immediate repeat:** Advance with $\#pool \geq 2$ gives $\text{playing}' \neq \text{playing}$ (equivalently $\text{playing}' \neq \text{lastPlayed}'$).
- **Sole looping case:** Advance replays only when $\#pool = 1$, the transient before the second track lands.

9 Findings from Formalisation

Writing the model surfaced these decisions, which the informal design (mission m-2026-07-04-018) must resolve consistently:

1. **Skip while generating the first track is undefined and is excluded.** `Advance` requires `playingFilling` or `playingRotating`; in `generatingFirst` nothing is playing, so `/music next` must be a no-op until track #1 lands.
2. **Auto-advance and manual skip are the same transition** ($\text{TrackEnds} \hat{=} \text{Skip} \hat{=} \text{Advance}$). The implementation should share one code path, not two.
3. **TurnOn, VibeChange, and restart share one “enter a pool” rule** keyed on $\#pool$ (empty $\rightarrow$ generate first; partial $\rightarrow$ play + fill; full $\rightarrow$ play + rotate). This is a natural single unit to extract.
4. **VibeChange abstracts over “finish current first.”** The model captures the switch after the current song ends; the temporal “finish current” is a refinement realised by deferring the switch to the next track-end.
5. **The only looping is the single-track transient.** Any other repeat (the shipped bug) violates the *No immediate repeat* property and is therefore a defect, not a mode.
6. **Fresh-track precondition ties to naming.** `FillProduces` requires $\text{newTrack?} \notin \text{pool}$; this is only safe because `bas7`’s deterministic naming guarantees uniqueness.